Project 1

Enterprise Static Asset CDN

Complete Build, Troubleshooting, Validation & Destruction Runbook

Scope: Terraform + Amazon S3 + CloudFront + Origin Access Control (OAC) + AWS WAF + GitHub Actions + GitHub OIDC + remote Terraform state

Purpose: This document records the practical sequence followed during the project, including commands, configuration concepts, validation tests, GitHub Actions/OIDC failures, fixes, and the final AWS cleanup performed to avoid ongoing charges.

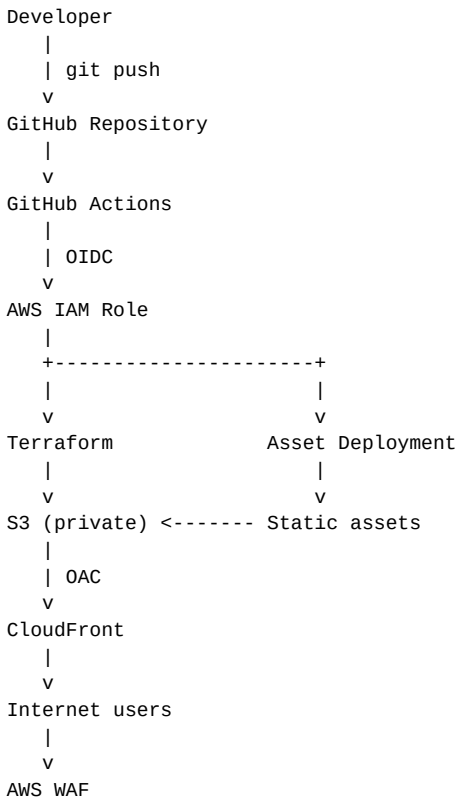
> **Important:** This is a learning/runbook document. Account IDs, repository names, resource names and example values are included where useful, but credentials/secrets must never be committed to Git.

Contents

1. Project Objective

Build an enterprise-style static asset delivery platform where static assets are stored privately in Amazon S3 and delivered globally through Amazon CloudFront. Terraform manages the AWS infrastructure; GitHub Actions deploys infrastructure and assets; AWS IAM OIDC removes the need for long-lived AWS access keys in GitHub Actions.

Target architecture



> Core security design: users do not receive public S3 access. CloudFront uses Origin Access Control (OAC) to access the private bucket.

2. Initial Terraform Project Setup

The project was created locally as an enterprise-static-asset-cdn repository with Terraform under the terraform/ directory and GitHub workflow files under .github/workflows/.

```
# Typical project structure
enterprise-static-asset-cdn/
■■■ .github/
■   ■■■ workflows/
■       ■■■ terraform.yml
■■■ assets/
■   ■■■ css/
■   ■■■ js/
■   ■■■ image/
■   ■■■ pdf/
■■■ terraform/
■   ■■■ backend.tf
■   ■■■ cloudfront.tf
■   ■■■ iam.tf
■   ■■■ github-actions-iam.tf
■   ■■■ main.tf
■   ■■■ outputs.tf
■   ■■■ providers.tf
■   ■■■ variables.tf
■■■ README.md
```

Terraform validation workflow used repeatedly:

```
terraform fmt
terraform validate
terraform plan
terraform apply
```

The Terraform plan/apply cycle produced successful resource creation, including examples such as **Plan: 3 to add, 0 to change, 0 to destroy** and **Apply complete! Resources: 3 added, 0 changed, 0 destroyed.**

3. S3 + CloudFront + OAC

The S3 bucket was configured as the private origin for the static assets. CloudFront was configured with an S3 origin, Origin Access Control (OAC), compression, a cache policy, HTTPS redirect behavior, and the default CloudFront certificate.

Important CloudFront configuration concepts used

• Origin domain: S3 regional domain name.

• Origin Access Control type: s3.

• Signing behavior: always.

• Signing protocol: sigv4.

• Allowed/cached methods: GET and HEAD.

• Viewer protocol policy: redirect-to-https.

• Price class: PriceClass_100.

• Default CloudFront certificate initially used for HTTPS.

• AWS WAF Web ACL attached to the CloudFront distribution.

A CloudFront domain was obtained and successfully tested. During the working phase, the distribution used the domain d3r6bijanzufr4.cloudfront.net.

A key Terraform lesson: a CloudFront OAC is not enough by itself. The S3 bucket policy must explicitly allow the CloudFront service principal to read the bucket using the appropriate SourceArn/SourceAccount conditions.

4. Terraform Outputs and the 'No outputs found' Message

At one point, running terraform output produced:

```
Warning: No outputs found

The state file either has no outputs defined, or all the defined
outputs are empty.
```

This was not an AWS failure. It meant the Terraform configuration did not yet expose the required values through output blocks. Outputs were then used to make the GitHub Actions workflow able to retrieve values such as the S3 bucket name and CloudFront distribution ID.

5. GitHub Repository and Remote Workflow

The project repository was pushed to GitHub. The workflow file was stored at:

```
.github/workflows/terraform.yml
```

The repository used the main branch. Git status and commit history were used throughout the project to track changes.

```
git status
git add .
git commit -m "meaningful change description"
git push origin main
git log --oneline
```

Important operational lesson: closing Git Bash does not delete the Git repository. Git history is stored in the .git directory and pushed commits are stored in GitHub.

6. Remote Terraform State

A dedicated S3 bucket was created for Terraform remote state. The project used a bucket similar to:

```
enterprise-static-cdn-sai-tfstate-2026
```

The state bucket was intentionally separate from the application asset bucket. The backend configuration was committed as terraform/backend.tf.

A Git status check showed backend.tf as untracked:

```
On branch main
Your branch is up to date with 'origin/main'.

Untracked files:
  terraform/backend.tf
```

The fix was to add and commit the backend file:

```
git add terraform/backend.tf
git commit -m "Configure remote Terraform state"
git push origin main
```

A Git warning about LF being replaced by CRLF was observed. This was a line-ending warning, not a deployment failure.

7. GitHub Actions → AWS Using OIDC

The project used GitHub Actions OIDC so the workflow could assume an AWS IAM role without storing a long-lived AWS access key and secret in GitHub.

The AWS side consisted of:

• GitHub Actions OIDC provider: token.actions.githubusercontent.com.

• IAM role: enterprise-static-cdn-github-actions.

• Trust policy action: sts:AssumeRoleWithWebIdentity.

• Audience condition: sts.amazonaws.com.

• Repository/branch/environment conditions restricting which GitHub tokens can assume the role.

The OIDC provider was verified with:

```
aws iam get-open-id-connect-provider \
  --open-id-connect-provider-arn \
  arn:aws:iam::<ACCOUNT_ID>:oidc-provider/token.actions.githubusercontent.com
```

The role trust policy was inspected with:

```
aws iam get-role \
  --role-name enterprise-static-cdn-github-actions \
  --query 'Role.AssumeRolePolicyDocument' \
  --output json
```

8. OIDC and IAM Errors — What Happened and How They Were Fixed

This was the most important troubleshooting section of the project. Several failures occurred because the GitHub Actions role did not initially have enough permissions to read IAM resources while Terraform was planning.

Error 1 — GetOpenIDConnectProvider denied

```
Error: reading IAM OIDC Provider:
AccessDenied: User ... assumed-role/enterprise-static-cdn-github-actions/GitHubActions
```

```
is not authorized to perform:
iam:GetOpenIDConnectProvider
```

Meaning: Terraform was running as the GitHub Actions role, but the role itself was not permitted to read the OIDC provider.

Fix: add the required IAM read permission to the GitHub Actions role policy, then commit/push the change and rerun the workflow.

Error 2 — ListRolePolicies denied

```
Error: reading inline policies for IAM role enterprise-static-cdn-github-actions:
AccessDenied ... iam:ListRolePolicies
```

Meaning: Terraform needed to inspect inline policies attached to the IAM role, but the assumed role lacked that read permission.

Fix: add iam:ListRolePolicies permission to the role's identity-based policy.

Error 3 — ListAttachedRolePolicies denied

```
Error: reading IAM Policies attached to Role (enterprise-static-cdn-github-actions):
AccessDenied ... iam:ListAttachedRolePolicies
```

Meaning: Terraform also needed permission to inspect managed policies attached to the role.

Fix: add iam:ListAttachedRolePolicies permission, commit, push and rerun the workflow.

These errors taught an important Terraform/IAM lesson: the AWS identity that runs Terraform needs both the permissions to manage resources and the read permissions Terraform requires to refresh/inspect those resources.

9. OIDC Token Validation Failure

After the IAM read-permission problems were fixed, the workflow reached AWS credential configuration but failed with:

```
Error: Could not assume role with OIDC:
The web identity token provided could not be validated.
See the AssumeRoleWithWebIdentity documentation for requirements.
```

The workflow repeatedly showed:

```
Run aws-actions/configure-aws-credentials@v4
Assuming role with OIDC
Assuming role with OIDC
...
Error: Could not assume role with OIDC
```

The trust policy was inspected. The configured provider was:

```
arn:aws:iam::<ACCOUNT_ID>:oidc-provider/token.actions.githubusercontent.com
```

The trust policy conditions included an audience of sts.amazonaws.com and repository/branch/environment subject values.

A key discovery was that the AWS account ID had been omitted in a recent change. The IAM/OIDC configuration was corrected to use the actual AWS account ID in the role ARN/resource references.

After the correction, GitHub Actions successfully configured AWS credentials and Terraform Apply completed.

10. GitHub Actions Terraform Workflow

The workflow eventually implemented a push-to-main deployment path and a separate static asset deployment job.

Conceptual workflow:

```
push to main
   |
   v
Terraform Apply
```

```
   |
   v
Deploy Static Assets
   |
   +--> terraform output -> S3 bucket name
   |
   +--> aws s3 sync assets/ s3://<bucket>/
   |
   +--> terraform output -> CloudFront distribution ID
   |
   +--> aws cloudfront create-invalidation --paths "/*"
```

The workflow used aws-actions/configure-aws-credentials@v4 with role-to-assume and the AWS region. It also used Terraform setup and initialization in the deployment job.

A successful workflow showed:

```
Terraform Apply       ■
Deploy Static Assets  ■
```

Terraform Plan was intentionally skipped on the push-to-main deployment path because the workflow condition selected the apply path for pushes to main.

11. GitHub Actions Queue / Stuck Run

One workflow run became stuck for approximately 42 minutes. The job page showed:

```
Evaluating terraform-apply.if
Evaluating: (success() && ((github.event_name == 'push') &&
(github.ref == 'refs/heads/main')))
Expanded: (true && (('push' == 'push') &&
('refs/heads/main' == 'refs/heads/main')))
Result: true
```

This proved the if condition evaluated successfully. The run was cancelled and a harmless empty commit was used to trigger a fresh workflow:

```
git commit --allow-empty -m "Trigger deployment workflow"
git push origin main
```

The new workflow succeeded. Lesson: distinguish workflow-condition evaluation from runner/job queue issues before changing Terraform or AWS configuration.

12. Static Asset Deployment

The project was extended so GitHub Actions uploaded static assets to S3 and invalidated CloudFront after deployment.

Asset deployment command used by the workflow:

```
aws s3 sync assets/ s3://<bucket>/ --delete
```

CloudFront invalidation:

```
aws cloudfront create-invalidation \
  --distribution-id "<DISTRIBUTION_ID>" \
  --paths "/*"
```

This created the intended continuous deployment flow: Git push → Terraform → S3 sync → CloudFront invalidation.

13. Asset Validation — CSS, JavaScript, Image and PDF

CSS test

The CSS file was changed and pushed. CloudFront returned the updated CSS after the deployment/invalidation workflow completed.

JavaScript test

```
console.log("Enterprise Static CDN - JavaScript loaded successfully");
```

The JS file was deployed and successfully retrieved through CloudFront.

Image test

```
assets/image/cdn-test.svg
```

The SVG was uploaded and successfully opened at:

```
https://<cloudfront-domain>/image/cdn-test.svg
```

PDF test

```
from reportlab.pdfgen import canvas
c = canvas.Canvas('assets/pdf/cdn-test.pdf')
c.drawString(100,750,'Enterprise Static CDN - PDF Test')
c.drawString(100,720,'Served through CloudFront + S3')
c.save()
```

The PDF was successfully opened through CloudFront at:

```
https://<cloudfront-domain>/pdf/cdn-test.pdf
```

14. PDF Creation Troubleshooting

First PDF attempt — Python not found

```
Python was not found; run without arguments to install from the Microsoft Store,
or disable this shortcut from Settings > Apps > Advanced app settings > App execution aliases.
```

Fix: Python was installed/configured, and ReportLab was made available.

Second PDF attempt — wrong working directory

```
FileNotFoundError: [Errno 2] No such file or directory:
'assets/pdf/cdn-test.pdf'
```

Cause: the terminal was in the home directory (~) instead of ~/enterprise-static-asset-cdn. The relative path therefore pointed to the wrong location.

Fix:

```
cd ~/enterprise-static-asset-cdn
mkdir -p assets/pdf
# then run the ReportLab command again
```

This successfully generated the PDF.

15. Security Validation

Direct S3 access test

```
curl -I https://enterprise-static-cdn-sai-2026-1.s3.us-east-1.amazonaws.com/css/style.css
```

Result: HTTP 403 / AccessDenied. This proved the S3 bucket was not publicly readable.

CloudFront access test

```
curl -I https://d3r6bijanzufr4.cloudfront.net/css/style.css
```

Result: HTTP 200. The same asset was available through the intended CloudFront path.

WAF test

```
curl -I "https://d3r6bijanzufr4.cloudfront.net/css/style.css?test=%3Cscript%3Ealert(1)%3C%2Fscript%3E"
```

The XSS-style request was used to verify WAF behavior. The project treated a blocked 403 response as the expected security outcome.

CloudFront WAF attachment was also checked through the CloudFront distribution configuration.

16. Project Destruction / Cost Cleanup

Because the project was a learning environment and the goal was to avoid ongoing billing, the AWS infrastructure was destroyed after testing.

Initial Terraform destroy failure — bucket not empty

```
Error: deleting S3 Bucket (enterprise-static-cdn-sai-2026-1):
StatusCode: 409
BucketNotEmpty: The bucket you tried to delete is not empty.
You must delete all versions in the bucket.
```

Cause: S3 versioning meant the bucket contained object versions and delete markers.

Fix: list versions/delete markers, generate a deletion manifest, delete all versions and markers, then delete the bucket.

```
aws s3api list-object-versions \
  --bucket enterprise-static-cdn-sai-2026-1 \
  --output json > versions.json

python -c "import json; d=json.load(open('versions.json')); o=[{'Key':x['Key'],'VersionId':x['VersionId']} for x in d.

aws s3api delete-objects \
  --bucket enterprise-static-cdn-sai-2026-1 \
  --delete file://delete.json
```

A JMESPath check initially failed because the CLI expression attempted to add two length() results and later failed when a missing field evaluated to None. The reliable verification used Python:

```
aws s3api list-object-versions \
  --bucket enterprise-static-cdn-sai-2026-1 \
  --output json > remaining.json

python -c "import json; d=json.load(open('remaining.json')); print('Versions:', len(d.get('Versions', []))); print('De
```

The application bucket was then deleted successfully and Terraform destroy completed.

17. Remote State Bucket Cleanup

The separate Terraform state bucket was still present after the main infrastructure destroy. It contained:

```
Versions: 28
Delete markers: 20
```

All versions and delete markers were deleted using the same S3 version cleanup technique. The state bucket was then deleted.

Important lesson: a resource created outside Terraform is not necessarily removed by terraform destroy. The Terraform state bucket was created separately and therefore required explicit cleanup.

18. Orphaned CloudFront Distribution — Final Cleanup

After the state bucket was deleted, an orphaned CloudFront distribution was discovered. A CloudFront list-distributions command showed:

```
Domain: d18l84wghcb993.cloudfront.net
ID:     E2I7Z1FL62V00Z
Status: Deployed
```

terraform state list then failed because the remote state bucket no longer existed:

```
Error loading the state:
S3 bucket "enterprise-static-cdn-sai-tfstate-2026" does not exist.
```

Therefore Terraform could no longer manage that state. The CloudFront distribution was disabled and then deleted directly through AWS CLI.

```
aws cloudfront get-distribution-config \
  --id E2I7Z1FL62V00Z \
  --output json > distribution-config.json

# Modify DistributionConfig.Enabled to false, then:
aws cloudfront update-distribution \
  --id E2I7Z1FL62V00Z \
  --if-match <ETAG> \
  --distribution-config file://distribution-config-disabled.json

aws cloudfront wait distribution-deployed \
  --id E2I7Z1FL62V00Z

aws cloudfront get-distribution-config \
  --id E2I7Z1FL62V00Z \
  --query ETag \
  --output text

aws cloudfront delete-distribution \
  --id E2I7Z1FL62V00Z \
  --if-match <NEW_ETAG>
```

The final get-distribution-config call returned NoSuchDistribution, confirming that the orphaned distribution was gone.

19. Terminal Paste-Control Error

While running the CloudFront wait command, terminal paste-control characters appeared before aws:

```
^[[200~aws cloudfront wait distribution-deployed ...
bash: $'\E[200~aws': command not found
```

Cause: bracketed-paste control characters were accidentally included in the pasted command. Fix: retype/run the command without the control characters.

20. Final Validation Checklist

Component / Test	Result
Terraform infrastructure creation	Completed
Private S3 origin	Completed
CloudFront + OAC	Completed
AWS WAF attached	Completed
GitHub repository	Completed
Remote Terraform state	Completed
GitHub Actions	Completed
AWS OIDC authentication	Completed after IAM/trust fixes
Static asset sync	Completed
CloudFront invalidation	Completed
CSS delivery	Verified
JavaScript delivery	Verified
Image delivery	Verified
PDF delivery	Verified
Direct S3 access	403 / blocked

Component / Test	Result
CloudFront access	200 / working
WAF malicious-request test	Blocked as expected
Application S3 bucket cleanup	Completed
Terraform state bucket cleanup	Completed
Orphaned CloudFront cleanup	Completed

21. Interview-Ready Explanation

How would you explain this project?

I built an enterprise-style static asset delivery platform using Terraform, Amazon S3, CloudFront, AWS WAF and GitHub Actions. The assets were stored in a private S3 bucket, and CloudFront used Origin Access Control with SigV4 to access the bucket. GitHub Actions authenticated to AWS using GitHub OIDC and an IAM role instead of long-lived AWS keys. On every push to main, Terraform applied the infrastructure and the deployment job synchronized assets to S3 and invalidated CloudFront. I validated CSS, JavaScript, image and PDF delivery through CloudFront, verified direct S3 access returned 403, and tested the WAF with an XSS-style request. I also troubleshot IAM read permissions, OIDC trust configuration, workflow queue behavior, S3 version cleanup and an orphaned CloudFront distribution during teardown.

Key interview questions this project prepares you for

• Why use CloudFront in front of S3?

• Why keep S3 private?

• What is CloudFront OAC and why use SigV4?

• How does GitHub Actions OIDC work with AWS IAM?

• What is the difference between an IAM trust policy and an identity policy?

• Why did Terraform need iam:GetOpenIDConnectProvider, iam:ListRolePolicies and iam:ListAttachedRolePolicies?

• Why can Terraform destroy fail on a versioned S3 bucket?

• How do you invalidate CloudFront after an asset deployment?

• How did you prove S3 was private?

• How did you test WAF?

• What happens when remote Terraform state is deleted before all resources are gone?

• How would you safely clean up an orphaned CloudFront distribution?

Most important lesson

> Terraform is declarative, but real infrastructure operations require understanding the AWS APIs, IAM permissions, state lifecycle, service-specific deletion behavior and CI/CD execution environment. The troubleshooting work in this project is as valuable for interviews as the initial successful deployment.

22. Command Cheat Sheet

```
# Terraform
terraform fmt
terraform validate
terraform plan
terraform apply
terraform destroy
terraform output
terraform state list

# Git
git status
git add .
git commit -m "message"
git push origin main
git log --oneline

# AWS S3
aws s3 ls
aws s3api head-bucket --bucket <bucket>
aws s3api list-object-versions --bucket <bucket> --output json
```

```
aws s3api delete-objects --bucket <bucket> --delete file://delete.json
aws s3api delete-bucket --bucket <bucket> --region us-east-1

# CloudFront
aws cloudfront list-distributions --output table
aws cloudfront get-distribution-config --id <distribution-id>
aws cloudfront update-distribution ...
aws cloudfront wait distribution-deployed --id <distribution-id>
aws cloudfront delete-distribution --id <distribution-id> --if-match <etag>

# IAM/OIDC
aws iam get-role --role-name <role>
aws iam get-open-id-connect-provider --open-id-connect-provider-arn <arn>

# HTTP validation
curl -I https://<cloudfront-domain>/css/style.css
curl -I https://<s3-domain>/css/style.css
```

End of Project 1 Runbook

The project was intentionally destroyed after validation to avoid unnecessary AWS charges. The GitHub repository and Git history can be retained as the project record even after the AWS infrastructure is removed.